

COORDINATE TRANSFORMS

AADITH WARRIER

ROBOTICS RESEARCH CENTER, IIIT-H

MAY 12, 2026

- Motivation
- Recap: Coordinate Systems
- Translation
- Rotations
- Homogenous Transformation Matrix
- Conventions
- Data Formats

MOTIVATION

Consider a robot with a *simple* time-of-flight (ToF) range sensor.

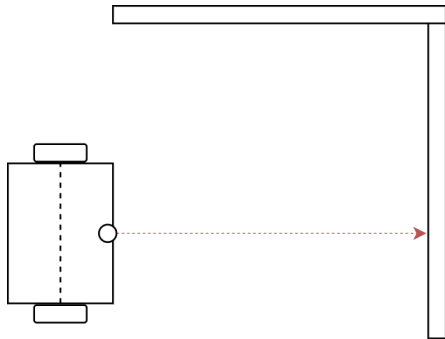


Figure: A differential drive robot equipped with a range sensor

MOTIVATION

Consider a robot with a *sophisticated* time-of-flight (ToF) range sensor.

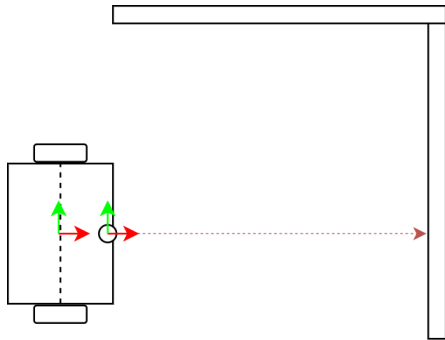


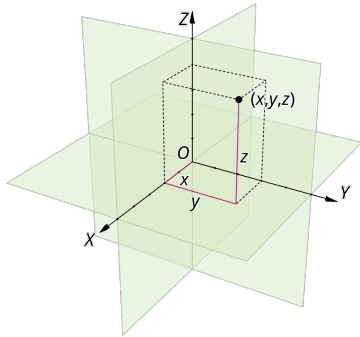
Figure: A differential drive robot equipped with a 2D LiDAR

There is a need to reconcile measurements that are taken in different "frames".

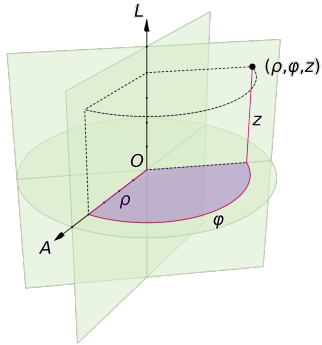
Expected Outcome:

Given a point represented in a coordinate frame A , ${}^A\mathbf{p}$. What is the same point expressed in another coordinate frame B , ${}^B\mathbf{p}$?

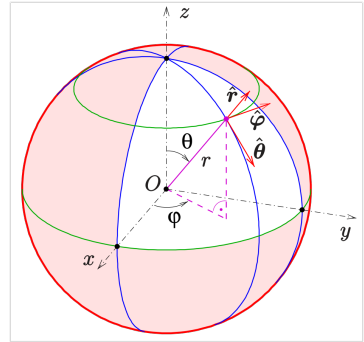
RECAP: COORDINATE SYSTEMS



Rectangular - $[x, y, z]$

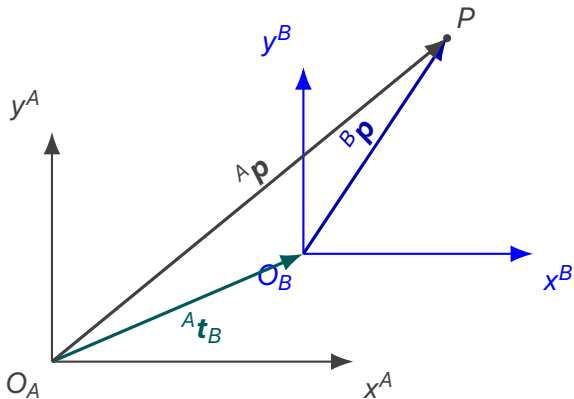


Cylindrical - $[\rho, \theta, z]$



Spherical - $[r, \theta, \phi]$

TRANSLATIONS

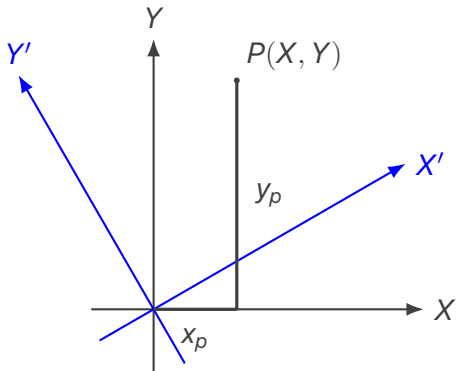


Pure translation means the axes are parallel, so the basis is the same.

$${}^A\mathbf{p} = {}^A\mathbf{t}_B + {}^B\mathbf{p}$$

where ${}^A\mathbf{t}_B$ is the vector from O_A to O_B expressed in frame A.

ROTATIONS - GEOMETRIC DERIVATION



Geometric Projection

By projecting the coordinates (x^B, y^B) directly onto the x^A and y^A axes using trigonometry:

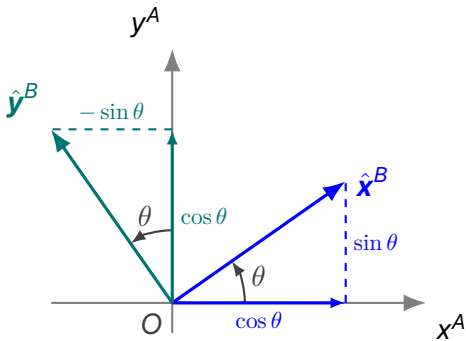
$$x^A = x^B \cos \theta - y^B \sin \theta$$

$$y^A = x^B \sin \theta + y^B \cos \theta$$

Writing this in matrix form:

$$\begin{bmatrix} x^A \\ y^A \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x^B \\ y^B \end{bmatrix}$$

ROTATIONS - LINEAR ALGEBRA DERIVATION



Algebraic view: projecting basis vectors

The unit axes of B expressed in A are:

$$\hat{\mathbf{x}}^B = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} \quad \hat{\mathbf{y}}^B = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}$$

Any point ${}^A\mathbf{p} = \begin{bmatrix} x^A \\ y^A \end{bmatrix}$ is a linear combination of these basis vectors:

$${}^A\mathbf{p} = x^B \hat{\mathbf{x}}^B + y^B \hat{\mathbf{y}}^B$$

Thus, the rotation matrix from B to A is:

$${}^A\mathbf{R}_B = \begin{bmatrix} \hat{\mathbf{x}}^B & \hat{\mathbf{y}}^B \end{bmatrix}$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

ROTATIONS - COMPOSITION

Rotation Matrices are part of the $SO(3)$ group

Matrix Multiplication Property Rotations are composed by multiplying matrices in the order of application (from right to left). For a sequence of rotations R_1 , then R_2 , then R_3 :

$$R_{total} = R_3 \cdot R_2 \cdot R_1$$

Important Note Matrix multiplication is **non-commutative** ($R_A R_B \neq R_B R_A$). The order of rotations significantly changes the final orientation.

Euler vs. Fixed Angles

- **Euler Angles:** Orientations are achieved through a sequence of rotations performed about the axes of the **moving (local) coordinate frame**.
- **Fixed Angles:** Orientations are achieved through a sequence of rotations performed about the axes of a **stationary (global) reference frame**.

ROTATIONS - EULER ANGLES

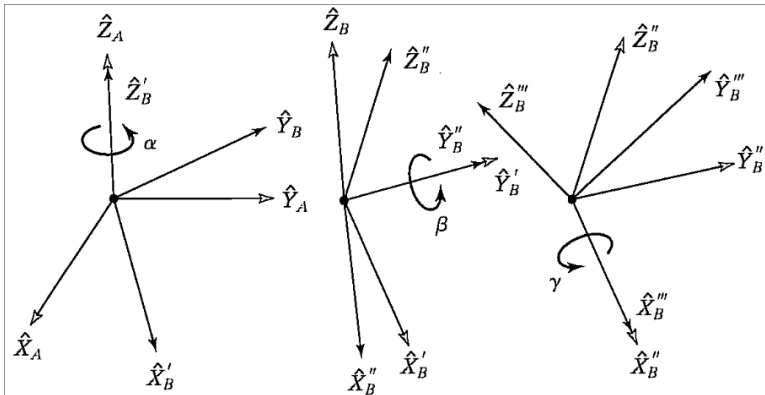


Figure: A sequence of rotations about the moving (local) axes.

$$\mathbf{R} = \mathbf{R}_Z(\alpha)\mathbf{R}_Y(\beta)\mathbf{R}_X(\gamma)$$

ROTATIONS - FIXED ANGLES

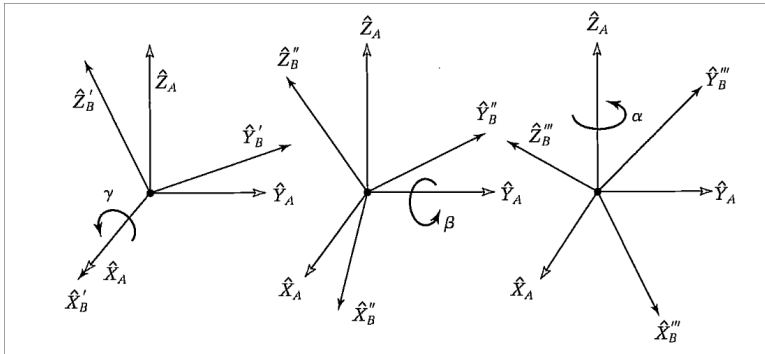


Figure: A sequence of rotations about the stationary (global) axes.

$$\mathbf{R} = \mathbf{R}_Z(\alpha)\mathbf{R}_Y(\beta)\mathbf{R}_X(\gamma)$$

Definition

- Gimbal lock occurs when two of the three rotation axes in a sequence become aligned, causing a loss of one degree of freedom.

When does it happen?

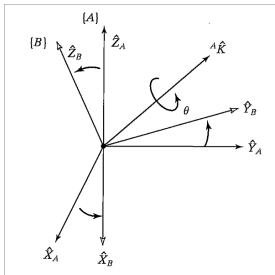
- Typically arises in Euler and Fixed angle representations.
- Most common in XYZ or ZYX sequences when the middle angle is near $\pm 90^\circ$.

Consequences

- The system can no longer rotate independently around all three axes.
- Control or interpolation becomes unstable or ambiguous.

AXIS-ANGLE REPRESENTATION

Rotations are described with respect to an axis (K) and an angle (θ).



$$\mathbf{R}_K(\theta) = \begin{bmatrix} C_\theta + k_x^2 V_\theta & k_x k_y V_\theta - k_z S_\theta & k_x k_z V_\theta + k_y S_\theta \\ k_y k_x V_\theta + k_z S_\theta & C_\theta + k_y^2 V_\theta & k_y k_z V_\theta - k_x S_\theta \\ k_z k_x V_\theta - k_y S_\theta & k_z k_y V_\theta + k_x S_\theta & \cos \theta + k_z^2 V_\theta \end{bmatrix}$$

$$S_\theta = \sin \theta, \quad C_\theta = \cos \theta, \quad V_\theta = 1 - \cos \theta$$

$$q = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) \\ k_x \sin\left(\frac{\theta}{2}\right) \\ k_y \sin\left(\frac{\theta}{2}\right) \\ k_z \sin\left(\frac{\theta}{2}\right) \end{bmatrix}$$

$$q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$$

Learn more about Quaternions from this 3Blue1Brown video.

HOMOGENEOUS TRANSFORMATION

Given any two arbitrary frames A and B, there exists a transformation between them that is a combination of rotation and translation.

$${}^B\mathbf{p} = {}^B\mathbf{R}_A {}^A\mathbf{p} + {}^B\mathbf{t}_A$$

To combine them into a single operation, we use **homogeneous coordinates**. A homogeneous coordinate is formed from a 3D point by adding a fourth coordinate which is set to 1. Now, **a point in a frame A**, is a 4D vector represented as:

$${}^A\mathbf{p} = [p_x \quad p_y \quad p_z \quad 1]^T$$

HOMOGENEOUS TRANSFORMATION

Now we can combine the rotation and translation operation into one matrix-multiplication operation.

$${}^B\mathbf{T}_A = \begin{bmatrix} {}^B\mathbf{R}_A & {}^B\mathbf{t}_A \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

And any point in Frame A can be transformed to Frame B by:

$${}^B\mathbf{p} = {}^B\mathbf{T}_A {}^A\mathbf{p}$$

Homogenous Transformation Matrices are part of the SE(3) group.

INVERSE TRANSFORMATIONS

$$({}^B\mathbf{T}_A)^{-1} = {}^A\mathbf{T}_B$$

Given a homogeneous transformation:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

Using properties of rotation matrices:

- $\mathbf{R}^{-1} = \mathbf{R}^\top$ (orthonormal)
- Translation inverts with rotated negative: $-\mathbf{R}^\top \mathbf{t}$

Therefore, the inverse is:

$$\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

COMPOSITE TRANSFORMATIONS

Transforms between frames can be chained by pre-multiplication.

$${}^D\mathbf{T}_A = {}^D\mathbf{T}_C {}^C\mathbf{T}_B {}^B\mathbf{T}_A$$

Local Tangent Plane

- **NED**: x points North, y points East, z points Down.
- **ENU**: x points East, y points North, z points Up.

Global (WGS 84)

- **Geodetic**: Coordinates expressed as Latitude (ϕ), Longitude (λ), and Altitude (h).
- **ECEF**: Earth-Centered, Earth-Fixed rotating with the Earth.

■ KITTI Format

- ▶ Flattened 3×4 rigid body transformation matrix per line.
- ▶ Format: `r11 r12 r13 tx r21 r22 r23 ty r31 r32 r33 tz`.
- ▶ Represents pose of the camera relative to the first frame.

■ TUM Format

- ▶ Pose represented by translation and unit quaternion.
- ▶ Format: `timestamp tx ty tz qx qy qz qw`.
- ▶ Common in visual-inertial SLAM evaluation datasets.

NOTATION & CONVENTIONS

- ${}^A\mathbf{p}$: coordinates of a point/vector expressed in frame A
- ${}^A\mathbf{R}_B$: rotation matrix that maps points from frame B to frame A
- ${}^A\mathbf{t}_B$: position of origin of frame B expressed in frame A
- ${}^A\mathbf{T}_B$: homogeneous transformation matrix from frame B to frame A
- x^A, y^A, z^A : scalar components of a vector along the x, y, z axes of frame A
- $\hat{\mathbf{x}}^A, \hat{\mathbf{y}}^A, \hat{\mathbf{z}}^A$: unit basis vectors of frame A

- Kevin M. Lynch, Frank C. Park: *Modern Robotics* (free online)
- John J. Craig: *Introduction to Robotics: Mechanics and Control*
- Ethan Eade: *Lie Groups for 2D and 3D Transformations* (notes)
- Barfoot: *State Estimation for Robotics* (SE(3), Lie theory)